Multi-Vehicle Chassis Control

Preface: Multi-vehicle cooperative control is built on the distributed network architecture of ROS 2. Within the same local area network and under the same `ROS_DOMAIN_ID`, the primary vehicle, follower vehicles, and the control terminal (virtual machine/PC) can achieve shared control or grouped control through topic remapping or independent namespaces. This section uses two vehicles (`robot1` and `robot2`) as an example to introduce two modes: shared keyboard control and independent gamepad distribution control.

---

1. Program Description

This feature enables simultaneous control of multiple vehicle chassis and robotic arms via keyboard or gamepad.

1.1 Functional Requirements

Using two vehicles as an example, each vehicle must satisfy the following requirements simultaneously:

1. **Same Local Area Network**: The two vehicles and the control terminal (virtual machine/PC) must be connected to the same 2.4G/5G Wi-Fi router, ensuring their IP addresses are on the same subnet and can ping each other;
2. **Same ROS Domain ID**: The `ROS_DOMAIN_ID` of both vehicles must be configured identically (the system default is usually 61):
    - **Raspberry Pi 5**: Enter the Docker container and modify the `ROS_DOMAIN_ID` in `/root/.bashrc`, then run `source ~/.bashrc` to refresh the changes;
    - **Orin / RDK X5**: Modify the `ROS_DOMAIN_ID` in `~/.bashrc` directly in the terminal, then run `source ~/.bashrc` to refresh the changes;

2. Control Modes and Startup Steps

The system provides two control modes:

- **Shared Keyboard Mode**: At the underlying layer, both vehicles remap their respective velocity topics to the global topic `/cmd_vel`, so that publishing velocity from a single keyboard makes all vehicles move synchronously;
- **Gamepad Distribution Mode**: Both vehicles retain their own independent namespaces (`/robot1/cmd_vel`, `/robot2/cmd_vel`), while the gamepad control node distributes commands to the topics of both vehicles simultaneously.

2.1 Mode 1: Shared Keyboard Control Mode

Step 1: Launch the Shared Control Chassis Driver on Both Vehicles

Run the following command in the terminal of **Vehicle 1 (robot1)**:

```
ros2 launch yahboomcar_multi A1p_bringup_multi_ctrl.launch.xml
robot_name:=robot1
```

The successful startup screen is shown below:

```
jetson@yahboom:~$ ros2 launch yahboomcar_multi A1p_bringup_multi_ctrl.launch.xml robot_name:=robot1
[INFO] [launch]: All log files can be found below /home/jetson/.ros/log/2026-09-04-20-31-16-278737-yahboom-1004591
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [Ackman_driver_A1P-1]: process started with pid [1004726]
[INFO] [imu_attitude_converter-2]: process started with pid [1004728]
[INFO] [ekf_node-3]: process started with pid [1004730]
[INFO] [static_transform_publisher-4]: process started with pid [1004732]
[INFO] [joint_state_publisher-5]: process started with pid [1004734]
[INFO] [robot_state_publisher-6]: process started with pid [1004736]
[static_transform_publisher-4] [WARN] [1788525076.989350230] []: Old-style arguments are deprecated; see --help for new-style argume
nts
[static_transform_publisher-4] [INFO] [1788525077.020508542] [robot1.imu_static_tf]: Spinning until stopped - publishing transform
[static_transform_publisher-4] translation: ('0.000000', '0.000000', '0.000000')
[static_transform_publisher-4] rotation: ('0.000000', '0.000000', '0.000000', '1.000000')
[static_transform_publisher-4] from 'robot1/base_link' to 'robot1/imu_link'
[robot_state_publisher-6] [INFO] [1788525077.106175547] [robot1.robot_state_publisher]: got segment robot1/Gripping
[robot_state_publisher-6] [INFO] [1788525077.106999698] [robot1.robot_state_publisher]: got segment robot1/arm1
[robot_state_publisher-6] [INFO] [1788525077.107031827] [robot1.robot_state_publisher]: got segment robot1/arm2
[robot_state_publisher-6] [INFO] [1788525077.107046547] [robot1.robot_state_publisher]: got segment robot1/arm3
[robot_state_publisher-6] [INFO] [1788525077.107056595] [robot1.robot_state_publisher]: got segment robot1/arm4
[robot_state_publisher-6] [INFO] [1788525077.107065748] [robot1.robot_state_publisher]: got segment robot1/arm5
[robot_state_publisher-6] [INFO] [1788525077.107109109] [robot1.robot_state_publisher]: got segment robot1/arm_base_Link
[robot_state_publisher-6] [INFO] [1788525077.107123413] [robot1.robot_state_publisher]: got segment robot1/base_footprint
[robot_state_publisher-6] [INFO] [1788525077.107134773] [robot1.robot_state_publisher]: got segment robot1/base_link
[robot_state_publisher-6] [INFO] [1788525077.107145078] [robot1.robot_state_publisher]: got segment robot1/camera_link
[robot_state_publisher-6] [INFO] [1788525077.107154390] [robot1.robot_state_publisher]: got segment robot1/laser_Link
[robot_state_publisher-6] [INFO] [1788525077.107164182] [robot1.robot_state_publisher]: got segment robot1/lbw_Link
[robot_state_publisher-6] [INFO] [1788525077.107172919] [robot1.robot_state_publisher]: got segment robot1/lfr_Link
[robot_state_publisher-6] [INFO] [1788525077.107181399] [robot1.robot_state_publisher]: got segment robot1/lfw_Link
[robot_state_publisher-6] [INFO] [1788525077.107190071] [robot1.robot_state_publisher]: got segment robot1/llink1
[robot_state_publisher-6] [INFO] [1788525077.107199415] [robot1.robot_state_publisher]: got segment robot1/llink2
[robot_state_publisher-6] [INFO] [1788525077.107208280] [robot1.robot_state_publisher]: got segment robot1/llink3
[robot_state_publisher-6] [INFO] [1788525077.107217048] [robot1.robot_state_publisher]: got segment robot1/rbw_Link
[robot_state_publisher-6] [INFO] [1788525077.107225880] [robot1.robot_state_publisher]: got segment robot1/rfr_Link
[robot_state_publisher-6] [INFO] [1788525077.107235576] [robot1.robot_state_publisher]: got segment robot1/rfw_Link
[robot_state_publisher-6] [INFO] [1788525077.107244025] [robot1.robot_state_publisher]: got segment robot1/rlink1
[robot_state_publisher-6] [INFO] [1788525077.107252537] [robot1.robot_state_publisher]: got segment robot1/rlink2
[robot_state_publisher-6] [INFO] [1788525077.107261465] [robot1.robot_state_publisher]: got segment robot1/rlink3
[joint_state_publisher-5] [INFO] [1788525077.880684446] [robot1.joint_state_publisher]: Waiting for robot_description to be publishe
d on the robot_description topic...
[imu_attitude_converter-2] [INFO] [1788525078.028747565] [robot1.imu_attitude_converter]: IMU attitude converter: imu/data_raw -> im
u/data, publish_tf=False
[Ackman_driver_A1P-1] [INFO] [1788525078.774281005] [robot1.driver_node]: A1 driver initialized: imu_link=robot1/imu_link, pub_hz=20
.0, arm_status_hz=5.0, cmd_vel_timeout=0.50s, pub_odom=True
```

Run the following command in the terminal of **robot2**:

```
ros2 launch yahboomcar_multi A1p_bringup_multi_ctrl.launch.xml
robot_name:=robot2
```

The successful startup screen is shown below:

```
jetson@yahboom:~$ ros2 launch yahboomcar_multi A1p_bringup_multi_ctrl.launch.xml robot_name:=robot2
[INFO] [launch]: All log files can be found below /home/jetson/.ros/log/2026-09-04-20-33-53-122009-yahboom-1016626
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [Ackman_driver_A1P-1]: process started with pid [1016747]
[INFO] [imu_attitude_converter-2]: process started with pid [1016749]
[INFO] [ekf_node-3]: process started with pid [1016751]
[INFO] [static_transform_publisher-4]: process started with pid [1016753]
[INFO] [joint_state_publisher-5]: process started with pid [1016756]
[INFO] [robot_state_publisher-6]: process started with pid [1016759]
[static_transform_publisher-4] [WARN] [1788525233.696150421] []: Old-style arguments are deprecated; see --help for new-style argume
nts
[static_transform_publisher-4] [INFO] [1788525233.720604993] [robot2.imu_static_tf]: Spinning until stopped - publishing transform
[static_transform_publisher-4] translation: ('0.000000', '0.000000', '0.000000')
[static_transform_publisher-4] rotation: ('0.000000', '0.000000', '0.000000', '1.000000')
[static_transform_publisher-4] from 'robot2/base_link' to 'robot2/imu_link'
[robot_state_publisher-6] [INFO] [1788525233.789704614] [robot2.robot_state_publisher]: got segment robot2/Gripping
[robot_state_publisher-6] [INFO] [1788525233.789973550] [robot2.robot_state_publisher]: got segment robot2/arm1
[robot_state_publisher-6] [INFO] [1788525233.790005071] [robot2.robot_state_publisher]: got segment robot2/arm2
[robot_state_publisher-6] [INFO] [1788525233.790018768] [robot2.robot_state_publisher]: got segment robot2/arm3
[robot_state_publisher-6] [INFO] [1788525233.790028432] [robot2.robot_state_publisher]: got segment robot2/arm4
[robot_state_publisher-6] [INFO] [1788525233.790037648] [robot2.robot_state_publisher]: got segment robot2/arm5
[robot_state_publisher-6] [INFO] [1788525233.790046448] [robot2.robot_state_publisher]: got segment robot2/arm_base_Link
[robot_state_publisher-6] [INFO] [1788525233.790056369] [robot2.robot_state_publisher]: got segment robot2/base_footprint
[robot_state_publisher-6] [INFO] [1788525233.790066001] [robot2.robot_state_publisher]: got segment robot2/base_link
[robot_state_publisher-6] [INFO] [1788525233.790075761] [robot2.robot_state_publisher]: got segment robot2/camera_link
[robot_state_publisher-6] [INFO] [1788525233.790085458] [robot2.robot_state_publisher]: got segment robot2/laser_Link
[robot_state_publisher-6] [INFO] [1788525233.790094706] [robot2.robot_state_publisher]: got segment robot2/lbw_Link
[robot_state_publisher-6] [INFO] [1788525233.790103698] [robot2.robot_state_publisher]: got segment robot2/lfr_Link
[robot_state_publisher-6] [INFO] [1788525233.790112402] [robot2.robot_state_publisher]: got segment robot2/lfw_Link
[robot_state_publisher-6] [INFO] [1788525233.790121363] [robot2.robot_state_publisher]: got segment robot2/llink1
[robot_state_publisher-6] [INFO] [1788525233.790130035] [robot2.robot_state_publisher]: got segment robot2/llink2
[robot_state_publisher-6] [INFO] [1788525233.790139347] [robot2.robot_state_publisher]: got segment robot2/llink3
[robot_state_publisher-6] [INFO] [1788525233.790148115] [robot2.robot_state_publisher]: got segment robot2/rbw_Link
[robot_state_publisher-6] [INFO] [1788525233.790157108] [robot2.robot_state_publisher]: got segment robot2/rfr_Link
[robot_state_publisher-6] [INFO] [1788525233.790165940] [robot2.robot_state_publisher]: got segment robot2/rfw_Link
[robot_state_publisher-6] [INFO] [1788525233.790174740] [robot2.robot_state_publisher]: got segment robot2/rlink1
[robot_state_publisher-6] [INFO] [1788525233.790183540] [robot2.robot_state_publisher]: got segment robot2/rlink2
[robot_state_publisher-6] [INFO] [1788525233.790192181] [robot2.robot_state_publisher]: got segment robot2/rlink3
[joint_state_publisher-5] [INFO] [1788525234.647242763] [robot2.joint_state_publisher]: Waiting for robot_description to be publishe
d on the robot_description topic...
[imu_attitude_converter-2] [INFO] [1788525234.792286975] [robot2.imu_attitude_converter]: IMU attitude converter: imu/data_raw -> im
u/data, publish_tf=False
[Ackman_driver_A1P-1] [INFO] [1788525235.625957231] [robot2.driver_node]: A1 driver initialized: imu_link=robot2/imu_link, pub_hz=20
.0, arm_status_hz=5.0, cmd_vel_timeout=0.50s, pub_odom=True
```

> [!NOTE]
> The `robot_name` parameter represents the unique namespace identifier of the current vehicle and defaults to `robot1`; the second vehicle is designated as `robot2`.

Step 2: Launch the Keyboard Control Node

Enter the following command in the terminal of any vehicle, or in the terminal of a control node on the same local area network:

```
ros2 run yahboomcar_ctrl yahboom_keyboard
```

Keyboard control instructions:

- `i` / `,`: Move forward / Move backward
- `j` / `l`: Turn left / Turn right
- `u` / `o`: Forward-left turn / Forward-right turn
- `m` / `.`: Backward-left turn / Backward-right turn
- `k` **or Space**: Emergency stop braking

In this mode, both vehicles jointly subscribe to the global `/cmd_vel` and `/arm_joint`, so pressing a key causes both vehicles to execute chassis movement and robotic arm actions synchronously.

2.2 Mode 2: Gamepad Distribution Control Mode

If you need to use a gamepad for wireless joystick control, the two vehicles' chassis do not need to remap topics to global topics, but instead maintain the independence of their respective namespaces.

> [!IMPORTANT]
> If Mode 1 was launched previously, first press `Ctrl + C` in each terminal to exit.

Step 1: Launch the Independent Namespace Chassis Driver on Both Vehicles

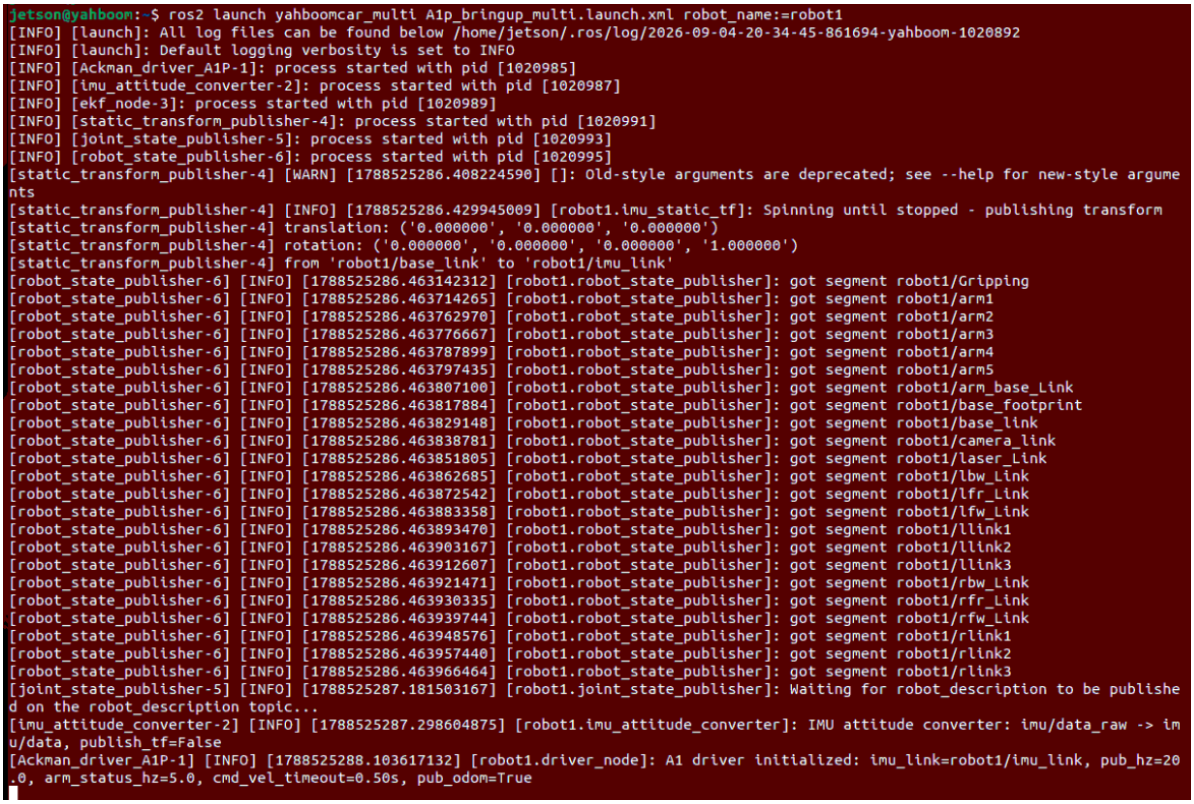
Run the following command in the terminal of **robot1**:

```
ros2 launch yahboomcar_multi A1p_bringup_multi.launch.xml robot_name:=robot1
```

Run the following command in the terminal of **robot2**:

```
ros2 launch yahboomcar_multi A1p_bringup_multi.launch.xml robot_name:=robot2
```

After successful startup, the terminal prints the readiness status of each sensor and the chassis driver:

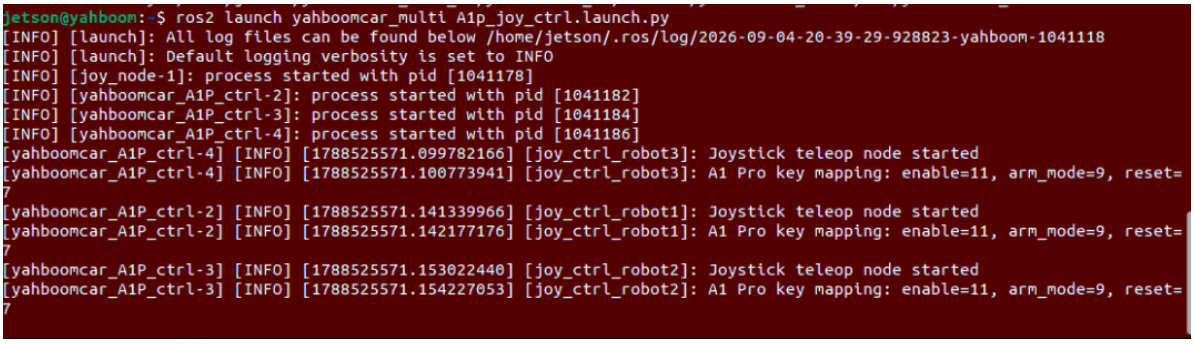


Step 2: Launch the Gamepad Control Node

Insert the USB gamepad wireless receiver into one of the vehicles or the control computer, then enter the following command in the terminal:

```
ros2 launch yahboomcar_multi A1p_joy_ctrl.launch.py
```

The terminal prints the gamepad node startup information:



In gamepad mode, `A1p_joy_ctrl.launch.py` internally generates control mapping nodes for `robot1`, `robot2` (and optionally `robot3`). The gamepad's joystick and buttons simultaneously broadcast control commands to `/robot1/cmd_vel`, `/robot2/cmd_vel`, as well as the buzzer, light, and robotic arm topics.

---

3. Multi-Device Communication and Node Topology Verification

3.1 View the Topic List

Run the following command in the terminal to view the currently active topics of the system:

```
ros2 topic list
```

You can see topics containing the dedicated namespaces of both vehicles (as well as global shared topics in keyboard mode):

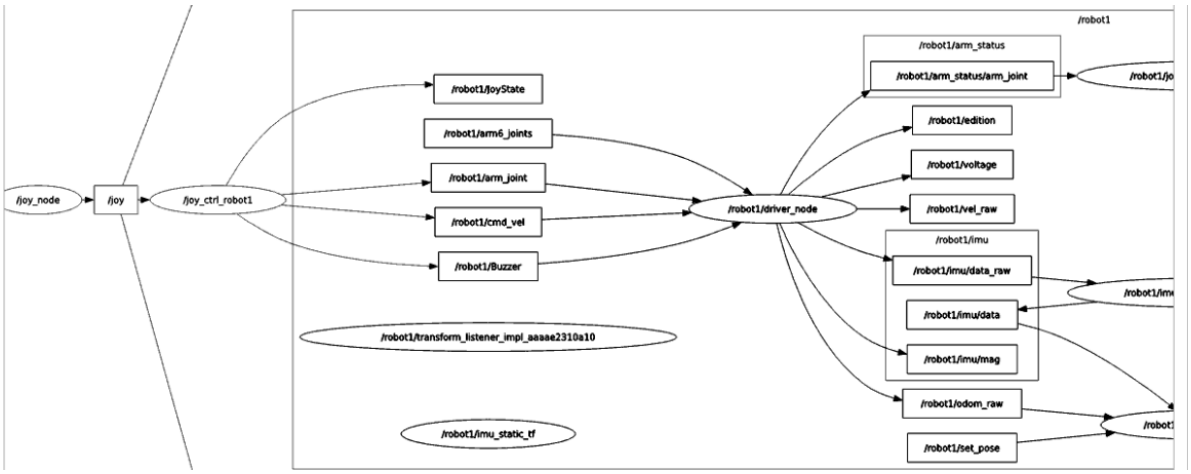
```
jetson@yahboom:~/yahboomcar_ros2_ws/yahboomcar_ws$ ros2 topic list
/diagnostics
/joy
/joy/set_feedback
/parameter_events
/robot1/Buzzer
/robot1/JoyState
/robot1/arm6_joints
/robot1/arm_joint
/robot1/arm_status/arm_joint
/robot1/cmd_vel
/robot1/edition
/robot1/imu/data
/robot1/imu/data_raw
/robot1/imu/mag
/robot1/joint_states
/robot1/odom
/robot1/odom_raw
/robot1/robot_description
/robot1/set_pose
/robot1/vel_raw
/robot1/voltage
/robot2/Buzzer
/robot2/JoyState
/robot2/arm_joint
/robot2/cmd_vel
/robot3/Buzzer
/robot3/JoyState
/robot3/arm_joint
/robot3/cmd_vel
/rosout
/tf
/tf_static
jetson@yahboom:~/yahboomcar_ros2_ws/yahboomcar_ws$
```

3.2 View the Multi-Device Computation Graph Topology

Run the rqt computation graph analysis tool in the terminal:

```
ros2 run rqt_graph rqt_graph
```

The computation graph visually shows the communication paths between each control node and the vehicle driver nodes:



- **Shared Keyboard Mode**: The control node publishes messages to the global topic `/cmd_vel`, while `/robot1/driver_node` and `/robot2/driver_node` listen to this topic simultaneously;
- **Gamepad Distribution Mode**: The gamepad node listens to `/joy` and publishes consistent velocity control commands to `/robot1/cmd_vel` and `/robot2/cmd_vel` in parallel.

---

4. Core Source Code Analysis

4.1 Shared Keyboard Chassis Launch File (A1p_bringup_multi_ctrl.launch.xml)

Source code path: `~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_multi/launch/A1p_bringup_multi_ctrl.launch.xml`

```
<launch>
    <arg name="robot_name" default="robot1"/>
    <group>
        <push-ros-namespace namespace="$(var robot_name)"/>
        <!-- 1. Low-level driver node: Ackermann steering chassis driver -->
        <node name="driver_node" pkg="yahboomcar_bringup"
exec="Ackman_driver_A1P" output="screen">
            <param from="$(find-pkg-share
yahboomcar_bringup)/param/a1_driver.yaml"/>
            <param name="imu_link" value="$(var robot_name)/imu_link"/>
            <param name="odom_frame" value="$(var robot_name)/odom"/>
            <param name="base_footprint_frame" value="$(var
robot_name)/base_footprint"/>
            <param name="pub_odom" value="true"/>
            <param name="pub_odom_tf" value="false"/>
            <!-- Key remapping: remap velocity and robotic arm topics to global
topics without namespaces for unified single-node control -->
            <remap from="cmd_vel" to="/cmd_vel"/>
            <remap from="arm_joint" to="/arm_joint"/>
            <remap from="/arm_status/arm_joint" to="/$(var
robot_name)/arm_status/arm_joint"/>
            <remap from="/arm_driver/get_actual_arm_position"
                   to="/$(var robot_name)/arm_driver/get_actual_arm_position"/>
        </node>

        <!-- 2. IMU attitude conversion node -->
        <node name="imu_attitude_converter" pkg="yahboomcar_bringup"
exec="imu_attitude_converter" output="screen">
            <param name="imu_input_topic" value="imu/data_raw"/>
            <param name="imu_output_topic" value="imu/data"/>
            <param name="publish_tf" value="false"/>
            <param name="use_odom_position" value="false"/>
        </node>

        <!-- 3. EKF Extended Kalman Filter data fusion node -->
        <node name="ekf_filter_node" pkg="robot_localization" exec="ekf_node"
output="screen">
            <param name="odom_frame" value="$(var robot_name)/odom"/>
            <param name="base_link_frame" value="$(var
robot_name)/base_footprint"/>
            <param name="world_frame" value="$(var robot_name)/odom"/>
            <param from="$(find-pkg-share yahboomcar_multi)/param/A1p_ekf_$(var
robot_name).yaml"/>
            <remap from="odometry/filtered" to="odom"/>
        </node>

        <!-- 4. IMU static TF broadcasting -->
```

```
        <node name="imu_static_tf" pkg="tf2_ros"
exec="static_transform_publisher"
              args="0 0 0 0 0 0 $(var robot_name)/base_link $(var
robot_name)/imu_link"/>
    </group>

    <!-- 5. Load the robotic arm and vehicle body URDF description file -->
    <include file="$(find-pkg-share
yahboomcar_multi)/launch/A1p_description_multi.launch.py">
        <arg name="robot_name" value="$(var robot_name)"/>
    </include>
</launch>
```

Key Mechanism Analysis:

1. **Namespace Isolation**: Using `<push-ros-namespace namespace="$(var robot_name)"/>` adds the `robot1/` or `robot2/` prefix to all nodes in the group, resolving node name conflicts in multi-device networking;
2. **Global Topic Remapping**:
   - `<remap from="cmd_vel" to="/cmd_vel"/>` redirects the vehicle's private `robotX/cmd_vel` to the global `/cmd_vel`;
   - The keyboard control node only needs to publish to the global `/cmd_vel` to broadcast to both vehicles;
3. **Coordinate Frame Isolation**: The odometry frame `odom` and the base frame `base_footprint` are parameterized as `$(var robot_name)/odom` and `$(var robot_name)/base_footprint` respectively, ensuring that the TF tree trunks do not interfere with each other.

4.2 Gamepad Multi-Vehicle Distribution Launch File (A1p_joy_ctrl.launch.py)

Source code path: `~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_multi/launch/A1p_joy_ctrl.launch.py`

```
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node

def generate_launch_description():
    first_robot_arg = DeclareLaunchArgument('first_robot1',
default_value='robot1')
    second_robot_arg = DeclareLaunchArgument('second_robot2',
default_value='robot2')
    third_robot_arg = DeclareLaunchArgument('third_robot3',
default_value='robot3')

    first_robot = LaunchConfiguration('first_robot1')
    second_robot = LaunchConfiguration('second_robot2')
    third_robot = LaunchConfiguration('third_robot3')

    # 1. Hardware gamepad driver node, listens to /joy
```

```
    joy_node = Node(
        package='joy',
        executable='joy_node',
    )

    # 2. Control node that distributes gamepad commands to robot1
    robot1_joy_ctrl_node = Node(
        package='yahboomcar_multi',
        executable='yahboomcar_A1_ctrl',
        name='joy_ctrl_robot1',
        output='screen',
        remappings=[
            ('cmd_vel', [first_robot, '/cmd_vel']),
            ('Buzzer', [first_robot, '/Buzzer']),
            ('RGBLight', [first_robot, '/RGBLight']),
            ('JoyState', [first_robot, '/JoyState']),
            ('arm_joint', [first_robot, '/arm_joint']),
        ],
    )

    # 3. Control node that distributes gamepad commands to robot2
    robot2_joy_ctrl_node = Node(
        package='yahboomcar_multi',
        executable='yahboomcar_A1_ctrl',
        name='joy_ctrl_robot2',
        output='screen',
        remappings=[
            ('cmd_vel', [second_robot, '/cmd_vel']),
            ('Buzzer', [second_robot, '/Buzzer']),
            ('RGBLight', [second_robot, '/RGBLight']),
            ('JoyState', [second_robot, '/JoyState']),
            ('arm_joint', [second_robot, '/arm_joint']),
        ],
    )

    return LaunchDescription([
        joy_node,
        first_robot_arg,
        second_robot_arg,
        third_robot_arg,
        robot1_joy_ctrl_node,
        robot2_joy_ctrl_node,
    ])
```

Key Mechanism Analysis:

- This launch file starts two `yahboomcar_A1_ctrl` instances simultaneously;
- Both instances listen to the low-level `/joy` gamepad button messages, but map their control outputs to `robot1/cmd_vel` and `robot2/cmd_vel` respectively through `remappings`;
- This achieves precise, independent, and synchronized control of any number of vehicles on the local area network with a single gamepad, without remapping global topics.